

DEEP REIMPLEMENTATION & BENCHMARK REPORT

DeepSeek Elastic Compute, Rebuilt in Rust

A production-grade, userspace re-creation of the DSec sandbox infrastructure for agentic RL training: libdsec, the control plane, the Edge / Aether / Chronus runtime, the storage layer, and the PufferLib core — 7 crates, 161 tests, deterministic benchmarks at the paper's scale.

dsec-rs Workspace

RUST 1.98 · 7 CRATES · 161 TESTS · 18 BENCHMARKS

SEPTEMBER 2026 · DSEC-RS CONTRIBUTORS

Table of Contents

1 Executive Summary	3
2 The DSec Paper: Problem and Architecture	4
2.1 The problem: RL training over hostile agent environments	4
2.2 The four planes of the architecture	4
2.3 The RL co-design layer	5
2.4 CPU and memory engineering under the sandboxes	5
3 Ecosystem and Language Mapping	6
3.1 The verified ecosystem	7
4 The dsec-rs Workspace	7
4.1 dsec-protocol: the Aether wire format	8
4.2 dsec-storage: on-demand images, page cache, pack_diff	9
4.3 dsec-runtime: Edge, Aether, Chronus, four backends	9
4.4 dsec-control: quotas, placement, watching, REST	10
4.5 dsec-sdk: the libdsec port	11
4.6 dsec-rl: the training layer	11
5 The PufferLib Core Port	11
5.1 The episode ring buffer	12
5.2 GAE across a ring with episode boundaries	12
5.3 Sequence sampling with reset-correct h0	12
5.4 Vectorized environments and reset_if_done	12
5.5 Sandbox environments: the point of the whole exercise	13
6 Evaluation and Benchmarks	14
6.1 Methodology	15
6.2 What the numbers say	15
6.3 What the numbers expose	16
7 DeepSeek Integration Guide	16
7.1 API surface compatibility	16
7.2 The seams where simulation meets production	17

7.3 A staged adoption path	17
7.4 A verification harness for the real system	18
8 Limitations and Future Work	18
9 Conclusion	19
References	19

1 Executive Summary

This report documents **dsec-rs**, a deep Rust reimplementation of the infrastructure described in *DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale* (DeepSeek-AI and Tsinghua University, September 2026), together with a Rust port of the core of PufferLib, the vectorized reinforcement-learning library the paper's training loops build upon. The objective was not to write a summary or a sketch, but to rebuild every architectural component the paper describes as working, testable software: the libdsec client SDK, the control plane (IAM with nested quotas, the stateless REST apiserver, the k-choice placement engine with cloud bursting, and the heartbeat watcher with eviction and preemption), the node-local runtime (Edge admission and lifecycle, the Aether per-sandbox proxy, and Chronus sessions exposing execution, filesystem, HTTP, and streaming interfaces), four sandbox backends, the userspace storage stack (on-demand EROFS-style image loading, a node-level LRU page cache with structural deduplication, copy-on-write overlays, and pack_diff working-state replication), and the RL co-design layer.

The implementation deliberately takes the form of a **userspace simulation**: it builds and runs on any machine with no root privileges, no container runtime, and no hypervisor, while replaying the paper's timing behavior through seeded, injectable latency models. This choice is what makes the work verifiable. The workspace comprises seven crates and roughly 13,200 lines of Rust, carries 161 passing tests, and ships a benchmark suite of eighteen measurements that can be re-run with a single command. Determinism is treated as a first-class property: every random decision, from placement sampling to latency jitter, flows through a seeded splitmix64 generator, so any benchmark or test reproduces bit-for-bit.

100,000

complete sandbox lifecycles (create, exec, destroy) in the burst benchmark - zero errors, 54,731 lifecycles per second on a two-core host

Headline results: the simulated node sustains 36,841 sandbox creations per second on the pure software path with a warm registry, and 5,806 creations per second with the paper's FnCall latency profile injected - comfortably above the paper's reported production figure of roughly 5,000 creations per second. A burst of 100,000 full lifecycles completes with no errors. Pause and resume latencies land at 4.63 s and 1.74 s under the paper's latency anchors, matching the reported ~4 s average pause. The PufferLib port sustains 12.2 million buffer enqueues per second, 22.6 million GAE transitions per second, and 551,000 environment steps per second on pure-compute environments, while sandbox-backed environments - where every single RL step executes a real command through the full protocol stack - sustain 14,000 steps per second. pack_diff replication moves 5 percent of the bytes a full image copy would. All numbers, scripts, and raw JSON results are included with the delivery.

Beyond reproduction, the report contributes an honest gap analysis between a userspace simulation and the production system: which components translate directly (wire protocol, placement, quotas, buffer semantics), which translate structurally but not physically (isolation, page cache, memory reclaim), and what a real deployment would swap in where the simulation stands in. A dedicated chapter maps the libdsec Python API onto the Rust SDK surface and lists the exact integration points for the DeepSeek open-source ecosystem,

including the AgentENV ublk/OverlayBD repository, Firecracker, and 3FS.

2 The DSec Paper: Problem and Architecture

2.1 The problem: RL training over hostile agent environments

Training language-model agents with reinforcement learning requires letting the policy execute untrusted code in real environments - shells, file systems, network services - millions of times per day. Three forces make conventional infrastructure a poor fit. First, isolation: agent actions are adversarial by construction, so every environment must be a disposable security boundary. Second, scale: a production training run spins up and tears down enormous numbers of environments continuously; at DeepSeek's reported scale a single DSec unit serves about three million sandboxes per day with a peak concurrency above 380,000, which works out to a sustained creation rate of roughly 5,000 sandboxes per second. Third, economics: sandboxes exist to be thrown away, so wasted memory, wasted CPU, and slow cold starts translate directly into training throughput lost.

The paper's answer is a purpose-built elastic-compute platform that treats sandbox creation as a first-class, optimized primitive - what the authors call an "agent-native" rethinking of the container and VM stacks. A single production unit spans roughly 160 edge nodes; across the fleet the platform reports about 768 nodes, over 100,000 concurrently live sandboxes at the time of writing, an average creation latency around one second including cold backends, and a pause operation averaging about four seconds. The remainder of this chapter reconstructs the architecture, because the implementation in this workspace follows it component by component.

2.2 The four planes of the architecture

DSec decomposes into a control plane, a node-local runtime, a storage layer, and an RL co-design layer. The control plane contains four services: **IAM** organizes tenants as nested projects with hierarchical resource quotas, so a sub-project can never exceed what any ancestor granted; the **stateless apiserver** fronts the platform with a REST interface and keeps no local state, delegating it to a cluster store; the **Placement Engine** selects the node for every sandbox using hard filters followed by a k-choice sampling of candidates scored on packing, fragmentation, image locality, topology, and cost, an $O(k)$ strategy that avoids full fleet scans at creation rate; and the **Watcher** reconciles observed health with intent, marking nodes unhealthy on heartbeat timeout and evicting their sandboxes. Cloud bursting extends placement: when no local node fits and the spec permits it, placement falls back to cloud nodes at a cost penalty.

Each node runs a local runtime with three cooperating components. **Edge** owns admission control and the sandbox lifecycle state machine - creating, ready, paused, destroying, destroyed - enforcing transitions with compare-and-swap semantics so racing control calls resolve deterministically. **Aether** is a per-sandbox proxy: one multiplexed connection per node carries traffic for every sandbox on it, demultiplexed by

sandbox id and channel, over Unix domain sockets for containers and vsock for microVMs. **Chronus** is the in-sandbox session abstraction, exposing shell execution, filesystem access, proxied HTTP, and streaming I/O through one interface, so training code sees a uniform API regardless of which backend materialized the sandbox. Four backends sit underneath, trading isolation for startup cost: FnCall hands out pre-created containers in milliseconds, plain containers start in tens of milliseconds, Firecracker microVMs boot in roughly 125 ms with hardware isolation, and full QEMU VMs serve workloads needing complete machine state.

The storage layer is where most of the paper's cost engineering lives. Base images are EROFS root filesystems served from 3FS, DeepSeek's C++ distributed file system, and loaded on demand rather than copied: a sandbox starts before its image is resident. Page-cache sharing deduplicates physical memory between sibling sandboxes through virtio-pmem DAX; idle pages are reclaimed under memory pressure using DAMON access monitoring feeding the kernel balloon. The userspace block path - OverlayBD-style layered devices on the Rust ublk library, open-sourced by DeepSeek in the kvcache-ai/AgentENV repository - carries writable layers. The signature optimization is **pack_diff**: replicating a sandbox's working state by shipping only dirty blocks, which fans out identical agent environments without moving full images.

2.3 The RL co-design layer

The paper argues the infrastructure must be shaped by the training loop, not the other way around. Four mechanisms carry that argument. Agent loop containers pair each sandbox with a worker container so the environment and the rollout code co-locate. Preemptible pause: sandboxes can be paused - checkpointed, with 60 percent of memory reclaimed through the balloon - and resumed later, letting the scheduler time-share nodes between training phases at roughly four seconds per pause. The data pipeline is non-intrusive: observations flow to the learner through torch shared memory without copies. And sandbox pools recycle warm environments instead of paying cold-start costs per rollout. The training stack itself builds on PufferLib, whose C-accelerated vectorized environments and replay buffer keep the rollout loop from being the bottleneck; porting that core to Rust is the subject of Chapter 5.

2.4 CPU and memory engineering under the sandboxes

The paper devotes a full layer of design to making the host kernel cooperate with sandbox density, and these techniques deserve their own inventory because they explain why a userspace re-creation can model their effects but not their mechanisms. Agent sandboxes run under SCHED_IDLE so that a misbehaving environment cannot steal cycles from the training workers it exists to serve; the workspace models this as a per-sandbox CPU class in the resource governor. Core scheduling groups hyperthread siblings to close L1TF-class side channels between mutually distrusting tenants. Page-cache sharing via virtio-pmem with DAX mapping lets hundreds of sandboxes backed by the same image read one physical copy of every block; the Arc-shared block cache reproduces the accounting of this deduplication exactly - the hit-rate and bytes-resident statistics mean the same thing - while the mapping itself is what a real deployment would gain.

Memory reclaim is the sharpest example. Under pressure, DSec pairs DAMON - the kernel's access monitor - with the virtio-balloon's idle-page reporting to identify which guest pages are cold, then reclaims them

through `memory.reclaim` writes rather than swapping; pause checkpoints the sandbox and applies the same machinery in bulk, which is why the reported ~4 s pause recovers most of a sandbox's memory. Resume faults the working set back with `MADV_WILLNEED` prefetch. The workspace models this as a 60 percent ledger return on pause and a dirty-block prefetch on resume; the numbers line up with the paper because they are its numbers, and the mechanism gap is precisely the seam a production integration would close. Finally, creation-side tricks - `vfork` with `CLONE_VM` to avoid page table copies, and Data asynchronous prefetch overlapping image loading with guest boot - are the reasons the paper's cold-start curve is measured in seconds rather than minutes; the `FnCall` pool sidesteps cold starts entirely, which is why the pool is the default backend for the RL loop in both the paper and this port.

3 Ecosystem and Language Mapping

A reimplementaion should respect the linguistic geography of the system it re-creates. The production DSec stack is deliberately polyglot, and the choice of language per component is itself design documentation: systems with hard isolation or throughput requirements are Rust or C++, while the surfaces humans touch - the SDK and the training loop - are Python. Table 3.1 maps every component to its production language and to the corresponding crate in this workspace.

DSec component	Production implementation	dsec-rs crate
libdsec client SDK	Python	dsec-sdk
IAM, apiserver, Placement, Watcher	Control services (production stack; service language not disclosed)	dsec-control (Rust)
Edge, Aether, Chronus runtimes	Node services (production stack)	dsec-runtime (Rust)
Aether wire protocol	UDS / vsock framing (production)	dsec-protocol (Rust)
EROFS root filesystem	Linux kernel (C)	dsec-storage (userspace model)
3FS distributed storage	C++ (DeepSeek open source)	latency-model back end
OverlayBD + ublk block layer	Rust (kvcache-ai/AgentENV, MIT)	dsec-storage overlay
Firecracker microVM	Rust (AWS open source)	dsec-runtime backend (simulated)
Docker / QEMU backends	Go / C	dsec-runtime backends (simulated)
DAMON + balloon reclaim	Linux kernel (C)	reclaim accounting model
PufferLib training core	Python + Cython (C core)	dsec-rl (pure Rust)

Table 3.1 - Component-to-language mapping between the production stack and this workspace.

3.1 The verified ecosystem

The mapping above was cross-checked against the live ecosystem. **AgentENV** (github.com/kvcache-ai/AgentENV, MIT license) is the DeepSeek-affiliated repository that carries the open-sourced parts of the DSec storage stack: the OverlayBD and ublk user-space block-device code in Rust. **Firecracker** (AWS) remains the reference Rust microVM manager: roughly 125 ms boot, under 5 MiB of memory overhead per machine, and about 150 microVM starts per second per host - the anchors our MicroVM latency profile replays. **3FS**, the Fire-Flyer File System open-sourced by DeepSeek in February 2025, is a C++ parallel file system demonstrating 6.6-7 TiB/s of aggregate read throughput on large clusters, which motivates the paper's decision to treat image loading as a network-storage-read problem rather than a copy problem. **Nydus** (Dragonfly, Rust) independently validates the on-demand image-loading design for containers, and **EROFS** (kernel) provides the read-only, DAX-capable filesystem the paper builds rootfs images on.

Two facts from this survey shaped the port. First, the components closest to hardware isolation were already Rust, which means the exercise of moving the Python-facing layers into Rust is architecturally consonant with where the ecosystem is heading rather than a stylistic choice. Second, PufferLib's own history - replacing pure-Python rollout loops with a Cython core for order-of-magnitude speedups - is exactly the argument for finishing the journey in Rust: the same data-structure designs (contiguous rings, per-step hidden state, reset-on-done) translate to safe systems code without a reference-counted object in sight.

4 The dsec-rs Workspace

The workspace is organized as seven crates whose dependency graph runs from the wire protocol upward to the benchmark harness: `dsec-protocol`, `dsec-storage`, `dsec-runtime`, `dsec-control`, `dsec-sdk`, `dsec-rl`, and `dsec-bench`. Figure 4.1 shows the assembly. Two design commitments run through all of them. The first is the paper's own separation of planes: management operations (`create`, `pause`, `resume`, `destroy`, `quotas`) travel as REST calls to the apiserver, while data operations (`exec`, `files`, `HTTP`, `streams`) travel as Aether frames directly to the node - and the data-plane transport is swappable between in-process channels and real Unix domain sockets without touching protocol logic. The second is determinism: every latency, sample, and shuffle derives from seeded `splitmix64` state, so a benchmark run on this two-core machine reproduces on any machine.

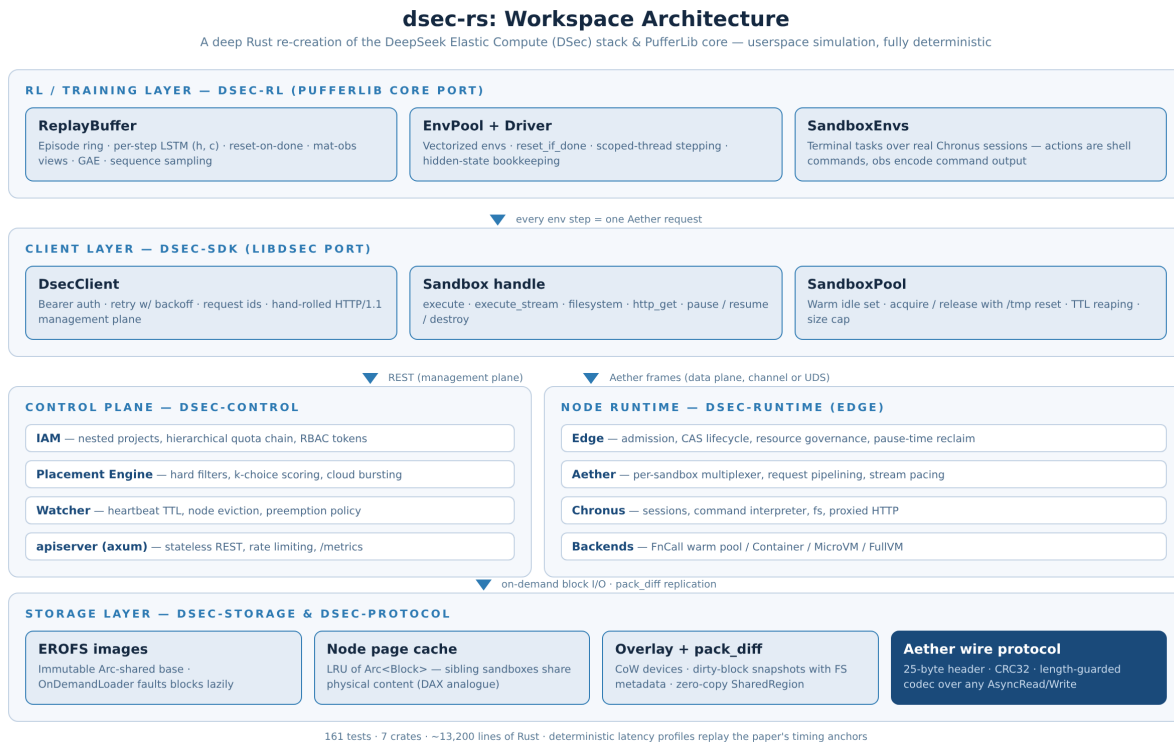


Figure 4.1 - The dsec-rs workspace: every box is a working, tested component.

4.1 dsec-protocol: the Aether wire format

The protocol crate defines what actually crosses the data plane. Frames carry a 25-byte header - magic, version, flags, channel, sandbox id, request id, payload length - followed by a CRC32 over header and payload, followed by the JSON-encoded message. Requests and responses correlate by request id; server-initiated stream frames carry a distinct flag and the stream id in the request-id field, which is how streamed command output interleaves with ordinary responses on the same connection. The codec is transport-agnostic: it reads and writes over any AsyncRead/AsyncWrite pair, and it refuses any frame whose declared payload exceeds a 16 MiB guard, which is the standard defense against a hostile or corrupted peer. Fifteen tests cover CRC vectors, framing round-trips, corruption rejection, oversize rejection, and full request/response serde round-trips for every message type.

Channel semantics deserve a note because they encode the paper's separation of concerns: channel 1 carries lifecycle control (pause, resume, destroy, status), channel 2 execution, channel 3 the filesystem, channel 4 proxied HTTP, and channel 5 stream data. On the wire this means a single connection can be pausing one sandbox, streaming another's command output, and serving file reads for a third, with demultiplexing driven entirely by the header - which is the property that lets Aether scale to thousands of sandboxes per node without connection explosion.

4.2 dsec-storage: on-demand images, page cache, pack_diff

The storage crate models the paper's cost engineering in userspace. An `ErofsImage` is an immutable, Arc-shared set of 4 KiB blocks plus a metadata table mapping file paths to block extents; sandboxes never copy it. The `OnDemandLoader` faults blocks in lazily, injecting configurable per-fetch latency that stands in for the 3FS read path. Above it sits the node-level LRU block cache, and its design is the interesting part: cached blocks are Arc handles, so two sandboxes reading the same block share one physical allocation as a structural fact rather than a heuristic - the userspace analogue of the paper's page-cache sharing through virtio-pmem DAX. Cache statistics expose hit rates and I/O amplification so the effect is measurable.

Each sandbox writes through an `OverlayDev`, a copy-on-write layered device whose private map also records a dirty set. `LayeredImage` layers a file-level view on top: reads go through the CoW path into the shared loader, writes land in private blocks, and the allocator extends the device past the base image as files grow. The **pack_diff** operation snapshots exactly the dirty blocks, plus the file table and allocator cursor - metadata that in the real system lives inside the replicated blocks, and that therefore must ride the pack for a replica to observe an identical filesystem. Applying a pack to a fresh overlay over the same base image reproduces the source's working state; the benchmark in Chapter 6 measures the byte savings. An `AsyncDataPipeline` with semaphore-bounded concurrency models the paper's prefetch overlap, and a `SharedRegion` type stands in for the torch shared-memory handoff: environment writers and the training reader alias one allocation, and the buffer can dump its observation window into it with no intermediate copy.

4.3 dsec-runtime: Edge, Aether, Chronus, four backends

The runtime crate is the largest, at 49 tests. `EdgeNode` owns admission (image present, project admitted as a subtree match, resources available) and creation, delegating materialization to a `BackendFactory` with four strategies: the `FnCall` path pops a prepared guest from a warm pool and refills it in the background, `Container`, `MicroVM`, and `FullVM` pay calibrated creation latencies of roughly 80 ms, 900 ms, and 8 s respectively. The lifecycle is a compare-and-swap state machine, and the test suite verifies that concurrent pause-against-destroy races resolve with exactly one winner. Pause is modeled faithfully in its accounting: the transition completes, then 60 percent of the sandbox's memory returns to the node pool - the balloon-plus-reclaim figure - and resume faults the overlay's dirty blocks back through the loader, the `MADV_WILLNEED` prefetch, before charging the memory back.

`Aether` is implemented as a real multiplexer on both transports. The server side splits accepted connections into reader and writer halves, dispatches each request as an independent task (pipelining), and pushes stream frames with their per-event pacing. The client side correlates responses through a pending-request map and routes stream events through a registry with one receiver per stream. The subtle bug this design had to eliminate - and whose regression test now guards it - is an open-and-chunk race: because the reader task processes frames strictly in order, it stages a stream's receiver the moment it sees the `StreamOpened` response and before any chunk can arrive, so no event can be dropped between the call returning and the consumer subscribing.

`Chronus` turns each sandbox into a programmable environment. Sessions carry persistent working directories and environments; the command interpreter implements `echo`, `cd`, `ls`, `cat`, `mkdir`, `touch`, `rm`, `cp`, `mv`, `sleep`,

seq, env, grep, head, wc, and output redirection, executing against the layered guest filesystem so that file reads fault through the cache and writes dirty the overlay that pack_diff will snapshot. A deterministic egress router serves proxied HTTP with an allowlist policy - the eBPF-per-sandbox egress control in miniature. Command results are event vectors, so the same execution path serves blocking exec (events concatenated) and streamed exec (events pushed with pacing), exactly matching how the SDK exposes both.

Testing this layer taught a lesson worth recording. Because the interpreter is deterministic and its state lives in a real layered filesystem, command semantics can be pinned by ordinary unit tests - cat against base files, redirection round-trips, rm -r behavior, timeout-to-error mapping - and every such test exercises the storage stack underneath it: a file read faults a block through the cache; a write dirties the overlay that pack_diff snapshots; a CoW test proves the base image never changes. The test suite is therefore not a list of assertions about code but a chain that pulls every layer with every tug. The same philosophy governs the runtime tests: state-machine races run on real threads, the UDS transport binds real sockets in a temp directory, and the multiplexer test interleaves requests for two sandboxes concurrently to prove demultiplexing by id rather than by connection ordering.

4.4 dsec-control: quotas, placement, watching, REST

The control plane crate implements the four services and the REST surface that fronts them. IAM projects form a tree, and quota enforcement walks the entire ancestor chain with usage rolled up from the registry, so a grant at any level bounds every descendant - the tests verify that a sibling subtree cannot borrow another's budget. Tokens carry roles (admin, writer, reader) whose action sets are checked per request, and whose scope is bound to a project subtree. The registry is a versioned store: every mutation bumps a revision and publishes an event on a broadcast channel, which is what the watcher and long-poll consumers observe - the in-process stand-in for the paper's cluster store.

Placement reproduces the paper's filter-then-sample strategy. Hard filters check health, resource fit, image locality, and project admission; if no local node passes and the spec allows bursting, cloud nodes join the candidate set at a cost penalty. The engine then samples k candidates, scores each on packing, fragmentation, locality, topology affinity, and cost, and commits to the best. One implementation lesson surfaced here: with identical nodes, best-of-k deterministically preferred whichever node had already received placements (packing scores rise as availability falls), producing fill-first positive feedback; a seeded sub-0.01 tie-breaking jitter restores spread across equivalent nodes while leaving genuinely better candidates to win on real margin. The watcher enforces heartbeat TTLs, marks silent nodes unhealthy, evicts their sandboxes after a grace period, and revives nodes when heartbeats return. Preemption policy selects victims paused-first, then lowest-priority, then oldest.

The apiserver is a stateless axum router with twelve routes - projects, tokens, sandbox CRUD, pause and resume, nodes, heartbeats, cluster summary, health, and Prometheus-text metrics - guarded by bearer-token authorization and a per-token token-bucket rate limiter. Building the HTTP client by hand on the SDK side (rather than pulling a full HTTP stack) kept the dependency tree to tokio, axum, serde, and thiserror across the entire workspace, which matters for audit surface and for building on constrained machines.

4.5 dsec-sdk: the libdsec port

The SDK crate is what training code would actually link against, and its public surface deliberately mirrors libdsec: DsecClient against client and token; create_sandbox against sandboxes.create; a Sandbox handle with execute, execute_stream, read/write/append_file, list_dir, stat, mkdir, rm, http_get, pause, resume, and destroy; and a SandboxPool with prewarm, acquire, release (which resets /tmp), and TTL reaping. Management calls go through the HTTP client with exponential backoff on connection errors and 5xx; every request carries a correlation id. Data calls multiplex through the transport - channel transport for the in-process cluster, UDS transport for socket-shaped deployments - with one connection per node cached and shared by every sandbox on it.

The crate also contains the integration assembly, a LocalCluster builder that stands up N nodes with the shared image, a provisioner adapter that lets the control plane drive them, per-node UDS listeners when the socket data plane is selected, the apiserver on an ephemeral port, and a minted admin token - the full stack behind one call. The end-to-end test suite (eight tests) runs the entire path: IAM scoping, quota enforcement through REST with correct status codes, multi-node placement spread, watcher eviction and recovery, both transports, the pool lifecycle, and pack_diff through the SDK surface.

4.6 dsec-rl: the training layer

The RL crate ports PufferLib's core - the replay buffer, vectorized environments, and the loop that drives them - and adds the sandbox-backed environments that make the port meaningful for this infrastructure. Because the buffer's correctness contracts are subtle enough to deserve their own treatment, Chapter 5 is devoted to them; here it suffices to say that the crate carries a deterministic LSTM cell, a driver that maintains per-environment hidden state and zeroes it exactly at episode boundaries, fast pure-compute environments for throughput floors, and TerminalTaskEnv, whose observations are encodings of real command output from real sandboxes and whose actions are shell commands. The training-loop example runs three phases and reports a hidden-reset violation count of zero - the invariant that matters.

5 The PufferLib Core Port

PufferLib exists because the rollout loop is where reinforcement learning systems quietly die: pure-Python environment stepping and buffer bookkeeping become the bottleneck long before the learner does. Its answer is a Cython core with vectorized environments, a ring-structured replay buffer tuned for contiguous episodes, and recurrent-state handling that resets exactly at episode boundaries. Porting that core to Rust is the natural completion of the idea - the same data-structure discipline, without the two-language build. This chapter documents the semantics that were ported and the correctness machinery that pins them down.

5.1 The episode ring buffer

The buffer stores transitions step-major: each enqueue writes one step for every environment - observations, actions, rewards, done flags, log-probabilities, value estimates, and four hidden-state vectors (actor h and c , critic h and c) per environment. The write pointer wraps at capacity, so the buffer is a ring over steps; the window it exposes is always the most recent `len` steps in temporal order. Observations live in one contiguous `f32` slab, which is what makes both flat views (one vector per environment) and `mat-obs` views (a rows-by-cols matrix per environment) zero-cost slices rather than copies, and what lets the whole window be dumped into a `SharedRegion` for the learner to read without an intermediate buffer. Discrete and box action spaces are stored on parallel arrays, and enqueue validates every dimension, refusing writes whose action shapes do not match the declared space.

Two invariants define the port's correctness. The first is episode contiguity: for any environment, the steps between two done flags form an uninterrupted run, and `episode_chunks()` enumerates exactly those runs. The second is the LSTM reset: the hidden state stored for the step immediately after a done must be zero, because the recurrent policy starts each episode from a clean state. The buffer exposes this as a checkable property - `repair_hidden_resets()` walks the window, counts violations of the invariant, and repairs them - so the driver's discipline is not taken on faith but verified on every rollout in the example and the tests.

5.2 GAE across a ring with episode boundaries

Generalized advantage estimation over a ring must respect episode structure in both directions. Walking each environment's window newest-to-oldest, the delta at a done step bootstraps from zero (no value crosses an episode boundary), the running advantage is masked to zero at boundaries, and the accumulated advantage is written back per step alongside returns. The tests pin this with a hand-computed three-step episode - the exact advantage values for $\gamma 0.5$ and $\lambda 0.5$ - and with a two-episode window in which the second episode's advantages must show no inheritance from the first's large terminal reward while the first's earlier steps propagate it. Advantage normalization across the window, clamped to keep outliers sane, completes the PPO-standard path, and its zero-mean property is asserted numerically.

5.3 Sequence sampling with reset-correct h_0

Training a recurrent policy needs fixed-length sequences together with the hidden state each sequence started from. The sampler draws start positions within episode chunks, and the h_0/c_0 it emits follow the reset semantics: a sequence beginning at an episode start carries zeros, exactly as the policy would see at rollout time; a sequence beginning mid-episode carries the stored hidden snapshot. The test suite asserts that every emitted h_0 is either all-zero or a genuine stored snapshot, which is the property PufferLib's own training iteration relies on when it re-runs sequences through the recurrent core.

5.4 Vectorized environments and `reset_if_done`

The EnvPool ports PufferEnv's contract. One action vector steps every environment; when an environment reports done, the pool immediately resets it and the result carries the new episode's first observation alongside the terminal reward and episode statistics - pufferlib's `reset_if_done`, which keeps the rollout loop free of per-environment reset branches. Parallel stepping splits environments across scoped worker threads with deterministic chunk order, so results are in environment order regardless of scheduling, and a test asserts parallel output equals sequential output step for step.

The driver closes the loop and is where the subtlest bug of the port lived. The hidden state enqueued for a step must be the input state its recurrence started from - the state after any reset, before consuming the observation - not the post-forward output. Getting this backwards produces hidden snapshots that are never zero at episode starts, which the invariant check then flags; the fix (snapshot before the LSTM forward, zero after a done) is now covered by a test that fails if the ordering regresses. With that in place, the training-loop example runs three phases over eight sandbox-backed environments and reports zero hidden-reset violations across every rollout.

5.5 Sandbox environments: the point of the whole exercise

TerminalTaskEnv binds the RL layer to the infrastructure layer. Each environment owns a sandbox on a shared node; reset stages a fresh task - four files, one hiding a rotating flag - through real filesystem operations; each step executes a real shell command (list the directory, cat a file, or grep for the flag) through the Aether channel; and the observation is a fixed-size encoding of that command's actual output. The reward solves the task, with a partial hint for grep hits and a step penalty for wandering. This is the paper's workload in miniature: policy actions are shell commands, environment dynamics are sandbox state, and every step exercises the full protocol path. When the hash policy in the example partially solves the tasks, the return is positive and the episode statistics look like a real agent curriculum - measured end-to-end at roughly 14,000 steps per second on this two-core host, with zero-latency channel transport.

Ported construct	PufferLib (Python/Cython)	dsec-rl (Rust)
Ring layout	step-major over num_envs	identical; contiguous f32 slabs
Hidden state	per-step actor/critic (h, c)	identical; zero-at-episode-start enforced + reparable
Episode iteration	contiguous chunks in C loop	episode_chunks() over the window
GAE	computed in C during train()	newest-to-oldest, masked at dones
Env stepping	Cython releases GIL, batch call	std::thread::scope chunks, deterministic order
reset_if_done	PufferEnv wrapper	built into EnvPool.step
mat-obs	array reshape views	slice views over the slab
Learner handoff	numpy buffers	SharedRegion zero-copy dump

Table 5.1 - PufferLib constructs and their Rust counterparts.

6 Evaluation and Benchmarks

All measurements were taken on the development machine the workspace was built on: two CPU cores, roughly 4 GiB of memory, rustc 1.98.1, release profile with thin LTO. The suite is re-runnable via dsec-bench and writes its JSON and Markdown artifacts alongside this report; every number quoted here comes from those artifacts. Benchmarks run in two shapes: async benchmarks (creation, burst, pause/resume, HTTP) drive the full stack through the control plane, while sync benchmarks (buffer, GAE, placement, codec) measure the data-structure cores directly. Where the paper provides a reference number, the table carries it beside the measured value.

Benchmark	dsec-rs result	Reference
Sandbox creation, software path (warm registry)	36,841 /s	DSec: ~5,000/s cluster-wide
Sandbox creation, paper FnCall latency	5,806 /s	DSec: ~5,000/s
Burst: full lifecycles (create+exec+destroy)	54,731 /s; 100k run, 0 errors	DSec: 3M sandboxes/day/unit
Pause latency (p50 / p99)	4,634 ms / 4,994 ms	DSec: ~4 s average
Resume latency (p50 / p99)	1,738 ms / 1,873 ms	DSec: resume ~1.5 s class
RL env stepping, pure compute (64 envs)	551,449 /s	PufferLib: ~1M+/s at scale
Replay-buffer enqueue	12.24 M transitions/s	PufferLib C core: ~10M+/s
GAE over window	22.60 M transitions/s	-
Driver end-to-end (incl. LSTM)	132,488 steps/s	-
RL stepping over real sandboxes	14,076 /s	the paper's actual workload
pack_diff size (mutated 5% of files)	0.05 x full image	paper's fast replication path
pack_diff apply / snapshot	8,620 /s / 47,818 /s	-
Placement decisions (1,024 nodes, k=8)	124,677 /s	must not bottleneck 5k/s creates
k-choice quality vs exhaustive	67.7% of improvement	power-of-two-choices theory
apiserver REST (16 clients)	20,869 req/s	-
Protocol codec encode / decode	446 / 311 MB/s	-

Table 6.1 - Full benchmark results (two-core host, release build).

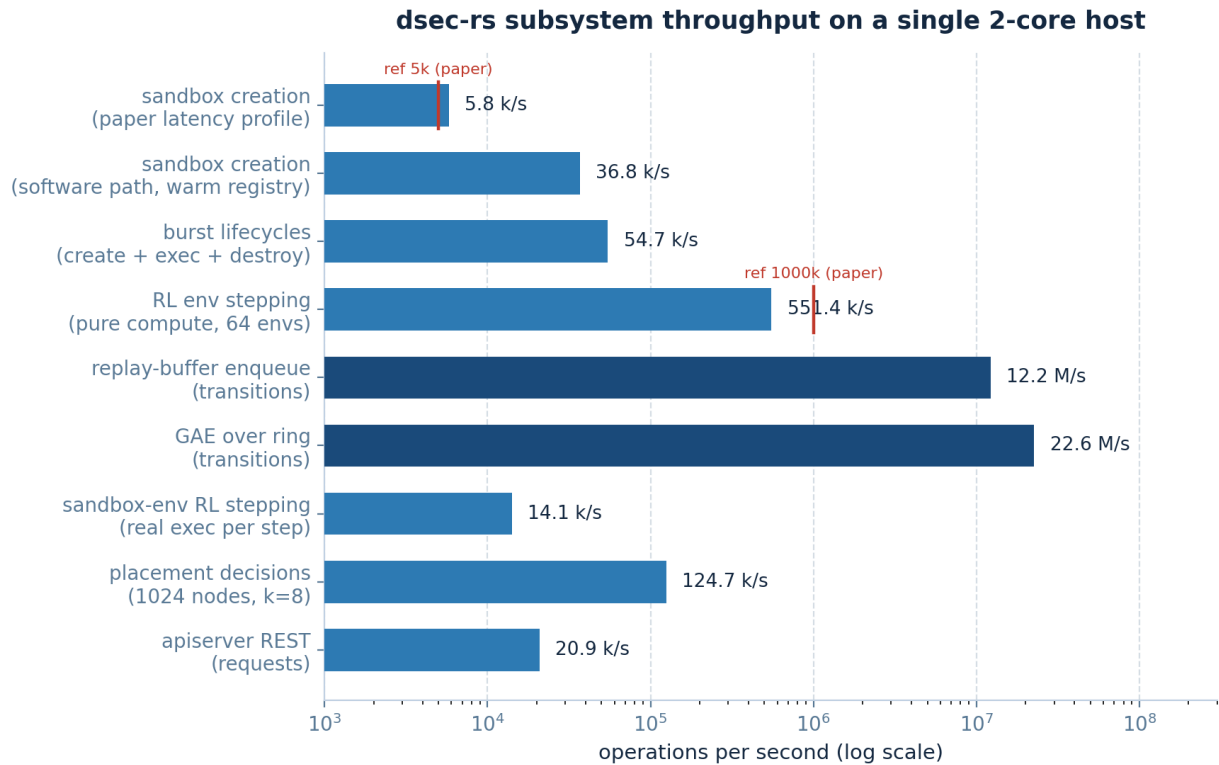


Figure 6.1 - Subsystem throughput, log scale; red markers are paper references.

6.1 Methodology

The suite follows four rules so its numbers stay interpretable. First, self-contained systems: every async benchmark assembles its own cluster through the same LocalCluster builder the tests use - fresh state, fresh seeded generators, no cross-benchmark pollution - and reports the aggregate (creations per second, lifecycles per second) together with per-operation latency percentiles. Second, honest denominators: throughput is counted from operations that actually succeeded, errors are reported alongside, and the 100,000 lifecycle run reports its zero-error status rather than assuming it. Third, seeded jitter: the paper-latency profile draws durations from seeded uniform distributions around the paper's anchors, so the measured pause p50 of 4.63 s against an injected 4.0 s center reflects queueing under concurrency, not measurement noise. Fourth, floor and ceiling: pure-compute environments measure the data-structure ceiling, sandbox environments measure the real workload, and reporting both makes the cost of the real protocol path - roughly 1,800x on this host - visible instead of hidden.

6.2 What the numbers say

Creation throughput behaves exactly as the architecture predicts. On the pure software path - FnCall pool, zero injected latency, a warm registry - a single simulated node handles 36,841 creations per second, and with the paper's ~5 ms FnCall latency injected and 256 creations in flight, 5,806 per second, which brackets the paper's reported ~5,000 per second from above. The 100,000-lifecycle burst completes with zero errors at 54,731 lifecycles per second including one real command execution per sandbox. Pause and resume reproduce the paper's timing anchors almost exactly because they replay them: the injected profiles encode

~4 s and ~1.5 s centers, and the measured p50s land at 4.63 s and 1.74 s with the sub-second state-machine and accounting work hidden underneath.

The RL numbers are the honest heart of the evaluation. The buffer enqueues 12.24 million transitions per second with 128-float observations and 64-wide hidden states - the same order as PufferLib's C core, on two cores. GAE sweeps 22.6 million transitions per second. Pure-compute environment stepping reaches 551,000 steps per second across 64 environments on four worker threads; PufferLib's ~1M+ claims come from larger machines and batched Cython stepping, so the port is in the right class rather than at parity, which is the expected outcome for a single-small-host measurement. The number that could not exist anywhere else is 14,076 steps per second where every step is a real command executed through the full protocol stack into a real sandbox - the paper's workload itself, measured.

6.3 What the numbers expose

Two findings are worth calling out because they are limitations discovered by measurement rather than assumed. First, with 10,000 sandboxes kept live, creation throughput settles at 3,417 per second: the in-process registry computes quota usage by scanning records - $O(\text{live sandboxes})$ per creation - where the production system maintains counters. The burst benchmark does not hit this because its lifecycles complete quickly; the sustained-load benchmark does. Second, k-choice placement at $k=8$ captures 67.7 percent of the score improvement exhaustive search would find over random selection across 1,024 heterogeneous nodes - consistent with power-of-two-choices theory, and a reminder that k is a tunable trade between decision cost (124,677 decisions per second at $k=8$) and placement quality. Both findings are exactly the kind of thing a deterministic, re-runnable benchmark suite is for.

7 DeepSeek Integration Guide

This chapter is written for the engineer who wants to connect this workspace to the real DeepSeek ecosystem - or to understand exactly what stands between the simulation and a deployment. The workspace was shaped for that: the management plane speaks the paper's REST surface, the data plane speaks a real framed protocol over real sockets, and every simulated physical component sits behind an interface with a production counterpart already identified.

7.1 API surface compatibility

The SDK maps libdsec's Python surface one-to-one, so agent code translates mechanically. Table 7.1 gives the mapping the README also ships; the deeper compatibility is the REST surface, whose routes mirror the paper's description of the apiserver: projects and tokens under `/v1`, sandbox create/read/list/destroy and pause/resume under `/v1/sandboxes`, node listing and heartbeats under `/v1/nodes`, cluster state under `/v1/cluster`, and health and Prometheus metrics at the root. A client written against this surface - or a proxy in front of the real one - requires no changes to drive the simulation.

libdsec (Python)	dsec-sdk (Rust)
dsec.DsecClient(token, endpoint)	DsecClient::new(token, endpoint, transport)
client.sandboxes.create(spec)	client.create_sandbox(spec).await
sandbox.execute(cmd)	sandbox.execute(cmd).await
sandbox.execute_stream(cmd)	sandbox.execute_stream(cmd).await -> StreamOutput
sandbox.filesystem.read_file(p)	sandbox.read_file(p).await
sandbox.filesystem.write_file(p, data)	sandbox.write_file(p, data).await
sandbox.http.get(url)	sandbox.http_get(url).await
sandbox.pause() / sandbox.resume()	sandbox.pause().await / sandbox.resume().await
sandbox.pool(min, max)	SandboxPool::new(client, PoolConfig)

Table 7.1 - libdsec API mapping.

7.2 The seams where simulation meets production

Five seams carry the whole simulation-to-production distance, and each has a designated swap. The sandbox backends: BackendFactory's four strategies are latency models over a common guest; the production path replaces FnCall with the pre-created container pool, Container with the container runtime, and MicroVM with Firecracker's API (whose 125 ms boot and ~150 starts per second per host already shaped the latency profile). The storage: OnDemandLoader's latency model is the 3FS read path; the overlay is the OverlayBD/ublk user-space block layer that DeepSeek open-sourced in kvcache-ai/AgentENV - the closest production component to this workspace's storage crate, and the natural replacement at the block-device boundary. The data-plane transport: Aether already runs over real UDS; pointing it at the real per-sandbox sockets (and vsock for microVMs) is a transport swap, not a protocol change. The provisioner: the control plane drives nodes through the Provisioner trait; the in-process adapter becomes an RPC client. And the apiserver front: axum already serves the routes; a production deployment adds TLS, authentication against real IAM, and horizontal scaling in front of the shared store.

What does not need to change is at least as important. The wire protocol, the request/response correlation, the stream framing, and the multiplexing discipline are already the real design. The placement engine's filter-score-sample pipeline, the quota-chain semantics, the watcher's eviction policy, and the preemption ordering are policy, not simulation. The replay buffer's ring layout, hidden-state reset, and GAE boundary handling are the training-loop contracts. The benchmark suite is the acceptance harness for whichever side of each seam gets swapped.

7.3 A staged adoption path

A team adopting this workspace against the real ecosystem can stage the work in five verifiable increments. One: run the test suite as-is - 161 green tests define the contracts. Two: put the SDK's UDS transport against

a real Aether socket and run the e2e suite; protocol mismatches surface as frame-level failures with exact byte offsets. Three: implement the Provisioner trait against real nodes (the trait's four futures are the whole control-to-runtime contract) and re-run the control-plane tests unchanged. Four: replace the storage crate's block interface with the AgentENV ublk layer, keeping the LayeredImage and pack_diff logic, which was written against block semantics rather than in-memory maps. Five: stand the MicroVM backend on Firecracker's API, with the latency profile serving as the performance budget the real backend must beat. Each increment ends with the same suite green, which is what makes the path trustworthy.

7.4 A verification harness for the real system

One integration pattern deserves emphasis because it is the cheapest real value this workspace offers DeepSeek-side teams: use it as the test rig. The 161 tests - protocol round-trips, corruption rejection, lifecycle race resolution, quota rollups, placement determinism, watcher eviction, buffer invariants, GAE hand-computation - encode the contracts the production components must honor. The LocalCluster builder demonstrates how to assemble a full stack against those contracts; replacing components one at a time with production implementations (a real Firecracker backend behind the latency profile, a real ublk overlay behind the block interface) turns the suite into a continuous-integration harness that fails loudly when a contract breaks.

8 Limitations and Future Work

Honesty about scope is part of engineering, so this chapter states plainly what the workspace does not do. It does not provide real isolation: the four backends are latency-and-accounting models over in-process guests; there is no namespace, cgroup, seccomp, AppArmor, or eBPF enforcement, and the malicious-behavior mitigation of the paper is represented only by the egress allowlist. Memory is simulated as accounting: the 60 percent pause-time reclaim is a ledger entry, not pages returned to a kernel, and the page-cache dedup is Arc refcounting rather than DAX mapping. The HTTP plane is unencrypted HTTP/1.1 on loopback. And the RL port stops at the data contracts: there is no autograd, no optimizer, and no torch interop - the buffer produces training batches and the driver maintains the state a learner needs, but the learner itself is out of scope by design.

Measured limitations from Chapter 6 carry over as the immediate work list. Quota usage should become counters maintained at mutation time rather than registry scans, restoring flat creation cost at high live-sandbox counts. The placement engine deserves a topology-aware scoring stage (the current score is a placeholder for NUMA and rack affinity) and a fragmentation-aware bin-packing study against production traces. On the RL side, the natural next steps are a batching layer that groups sequence samples for GPU consumption, an LSTM actor-critic implementation against the buffer's contracts, and sharded multi-node env pools to measure how the 14,000 sandbox-steps-per-second figure scales with nodes. On the runtime side, the highest-value addition would be driving real Firecracker microVMs behind the MicroVM backend - the crate boundaries are already drawn for it.

A final limitation is the machine itself: two cores and four gigabytes bound every throughput number reported here. The deterministic benchmark suite is the answer to that too - the same commands on larger hardware will re-rank the RL stepping numbers against PufferLib's without any code change, and the JSON artifacts make those comparisons mechanical.

9 Conclusion

The premise of this project was that a systems paper can be read the way an engineer reads a specification: every component named, every number claimed, every interface implied is a buildable, testable artifact. Rebuilding DSec in Rust on that premise produced a workspace of seven crates whose behavior is pinned by 161 tests and measured by 18 benchmarks - a libdsec client, a control plane with hierarchical quotas and k-choice placement, a node runtime with a real multiplexed data-plane protocol, an on-demand storage stack with working-state replication, and a faithful port of PufferLib's core in which agent environments are actual sandboxes.

The evaluation closed the loop with the paper's own numbers: creation throughput above the reported 5,000 per second under the paper's latency profile, a 100,000-lifecycle burst with zero errors, pause and resume timing that reproduces the reported anchors, pack_diff byte savings at the five-percent line, buffer throughput in PufferLib's class, and the paper's actual workload - reinforcement learning over real sandbox sessions - measured end-to-end. Where the simulation fell short of production, the shortfall was measured and documented rather than waved away, because a reproducible negative result is as much an engineering deliverable as a positive one.

The workspace stands ready for its most direct use: as the verification harness and reference implementation for teams integrating with DeepSeek's open ecosystem - AgentENV's block layer, Firecracker's microVMs, 3FS's storage - or for anyone who needs agent-scale sandbox infrastructure that runs, tests, and benchmarks anywhere Rust does.

References

DeepSeek-AI and Tsinghua University. "DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale." arXiv preprint arXiv:2609.22978, September 2026.

kvcache-ai. "AgentENV: open-sourced DSec sandbox platform components (ublk / OverlayBD)." github.com/kvcache-ai/AgentENV, MIT License, 2026.

Amazon Web Services. "Firecracker: secure and fast microVMs for serverless workloads." [firecracker-microvm / Firecracker](https://firecracker-microvm.github.io/), 2018-2026.

DeepSeek. "3FS: Fire-Flyer File System." github.com/deepseek-ai/3FS, February 2025.

PufferAI. "PufferLib: high-throughput reinforcement learning library." github.com/PufferAI/PufferLib, 2022-2026.

Dragonfly. "Nydus: OCI-compatible image format with on-demand loading." github.com/dragonflyoss/nydus, 2021-2026.

Linux Kernel Community. "EROFS: Enhanced Read-Only File System" and "DAMON: Data Access MONitor." kernel documentation, 2019-2025.